TransLink Transit Intelligence Pipeline

INFO 4190- PROJECT INTEGRATION 1 | FINAL REPORT

Group 9

Inderpreet Kaur 100383066 - Technical Design Lead
Drishti Kapoor 100435109 - Design & Documentation Lead

August 4, 2026

Table of Contents

1.0 Project Overview and Objectives 2
Refined Goals and Scope 2
Target Users 2
2.0 Midterm Improvements 2
3.0 Final System Design 3
Justification of Design Choices 10
4.0 Test Plan and Test Case Design 10
5.0 Peer Feedback and Improvements 15
6.0 Project Evaluation and Reflections 15
Progress Against Original Goals 15
Lessons, Team Dynamics, and Project Planning 16
Remaining Risks and Next Steps 16
7.0 Conclusion 16
8.0 Appendix 17

1.0 Project Overview and Objectives

Metro Vancouver's transit network, operated by TransLink, serves hundreds of thousands of daily commuters across buses, SkyTrain, SeaBus, and RapidBus. Existing tools such as Google Maps and TransLink's Trip Planner effectively answer real-time arrival questions; however, they provide limited support for understanding broader scheduled service patterns and identifying optimal departure windows. The TransLink Transit Intelligence Pipeline addresses this gap: an end-to-end system that automatically ingests TransLink's weekly GTFS data, processes it through an ETL pipeline, applies machine learning to identify service patterns, and presents insights through an interactive dashboard.

Refined Goals and Scope

Since the original proposal, three refinements were made based on professor feedback:

- K-Means clustering was reclassified as an optional stretch feature; the core value, schedule-frequency aggregation by route, stop, day, time, and direction, works without it.
- Language was standardized around "scheduled service frequency" and "service intensity," since the system cannot confirm actual passenger demand or crowding.
- User testing was added as a formal success metric for the plain-language departure recommendation.

Target Users

- Metro Vancouver daily commuters seeking smarter departure planning
- Transit researchers and urban planners studying network-wide service patterns
- Open data enthusiasts interested in civic technology applications

2.0 Midterm Improvements

Following the Midterm Report, feedback from the professor identified two small areas for improvement: a wording issue in the Literature Review and two minor professionalism/clarity issues. No new sources were added to the Literature Review, as the existing five sources were assessed as strong and well-integrated.

Section	Original Feedback	Improvements Made
Literature Review	The claim that scheduled service frequency is a "standard, accepted" proxy for service intensity should be worded more cautiously, and should clarify that frequency reflects service supply, not passenger demand or crowding.	Reworded the relevant sentence in the Roth et al. discussion to clarify that scheduled service frequency is a proxy for service supply, not actual ridership or demand.

Professionalism & Organization	Title page had a typo ("NFO 4190" instead of "INFO 4190"); some wording was informal ("crunching," "works fine without it"); the architecture diagram labelled the ML step "K-Means clustering on route behaviour" without noting it is optional, which could confuse readers on what is core versus stretch.	Corrected the title page typo. Replaced informal phrasing with more formal language. Updated the architecture diagram's ML box to explicitly label K-Means clustering as optional.

3.0 Final System Design

Design Artefacts

The final system design includes the following diagrams and mockups, carried forward from Progress Report 2 and finalized for this report:

- **Database ERD**: 12 tables (routes, trips, stops, stop_times, calendar, calendar_dates, shapes, transfers, directions, feed_info, route_clusters, etl_runs) with relationships and cardinality defined

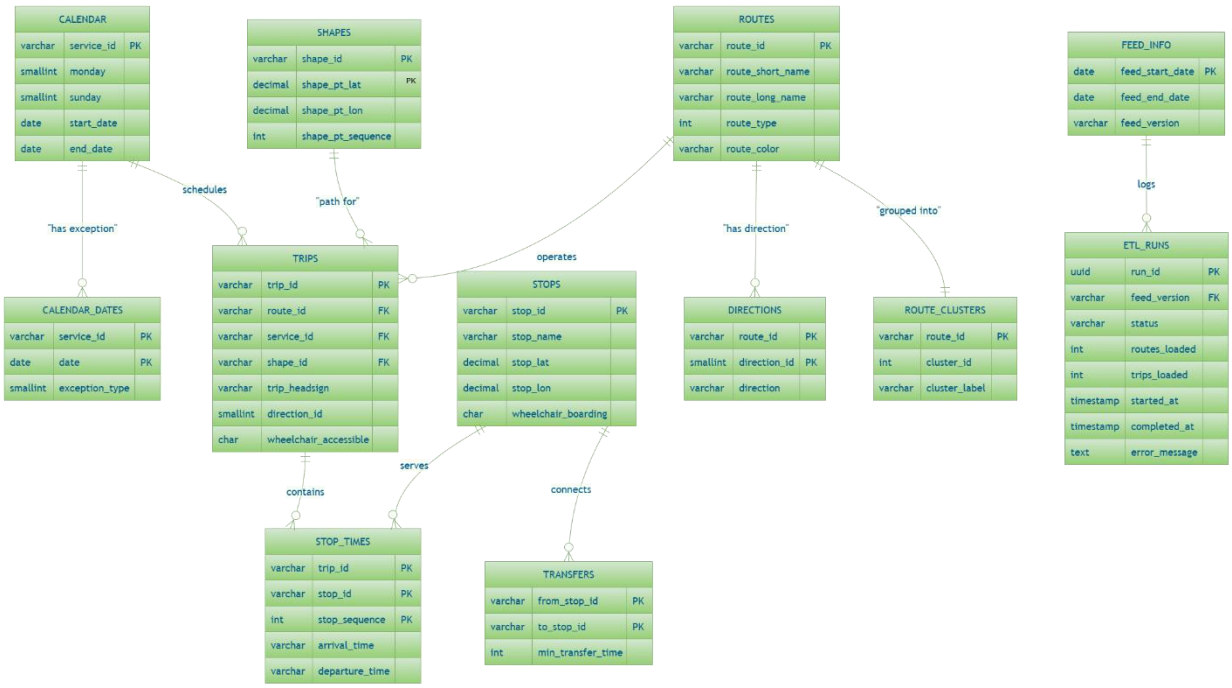


Figure 1: Database ERD

- **Domain Class Diagram**: core system entities (Route, Trip, Stop, StopTime, Calendar, CalendarDate, Shape, Direction, RouteCluster, Transfer, FeedInfo, EtlRun) and their relationships

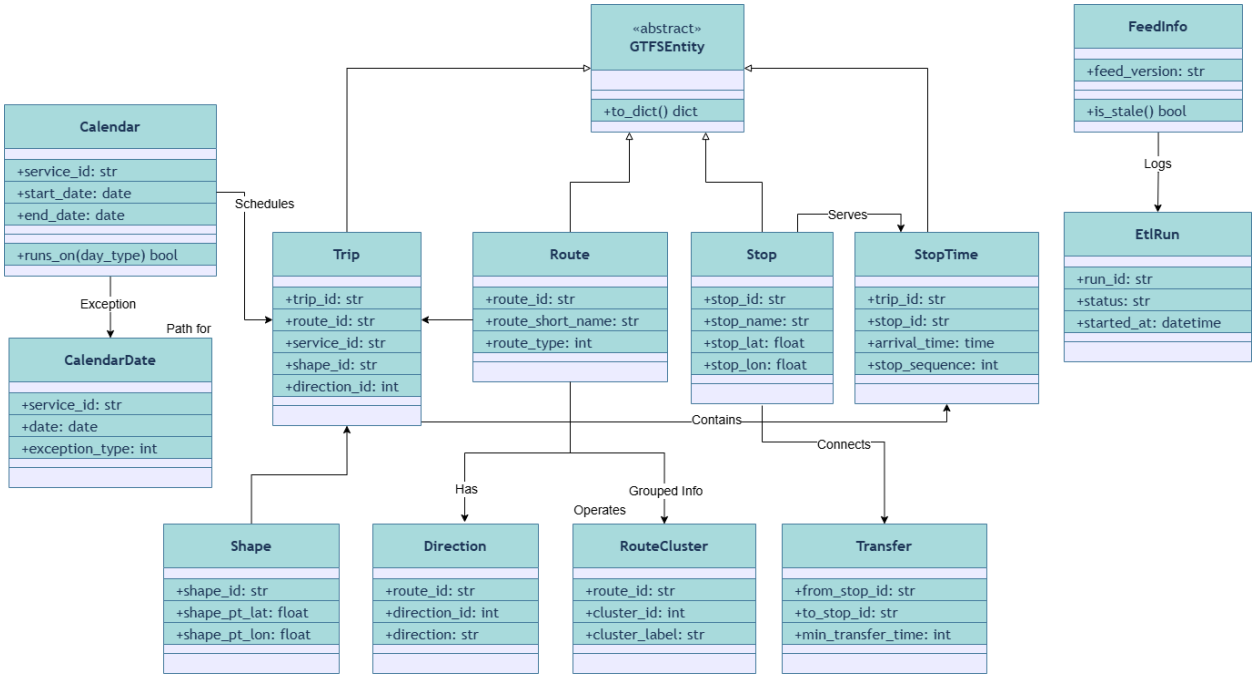


Figure 2: Domain Class Diagram

- **Service Importer Class Diagram**: the GTFSImporter component (download_feed, has_feed_changed, parse_files, validate, load_to_staging, swap_to_production, log_run)

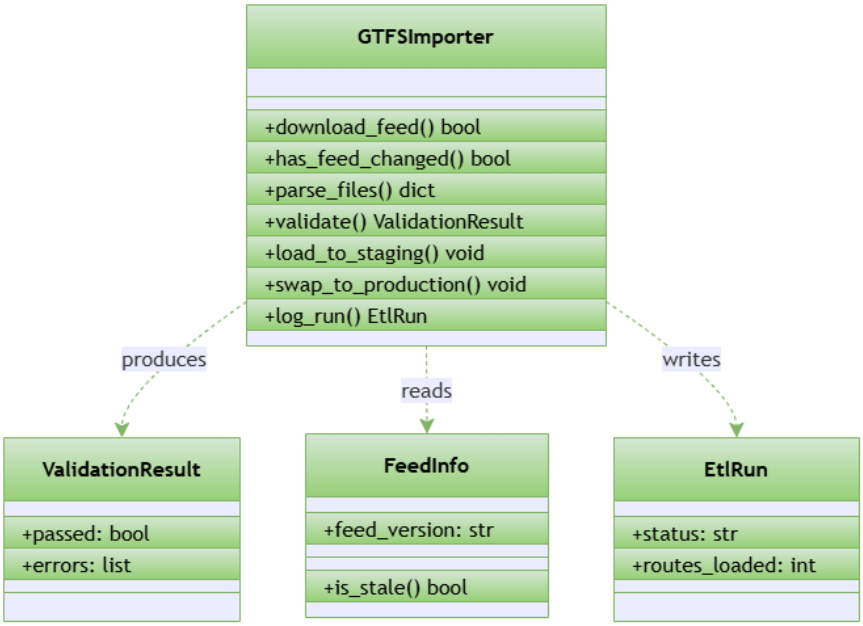


Figure 3: Service Importer Class Diagram

- **System Architecture Diagram**: four layers: Ingest, Process (ETL), Analyse (ML, optional), Present

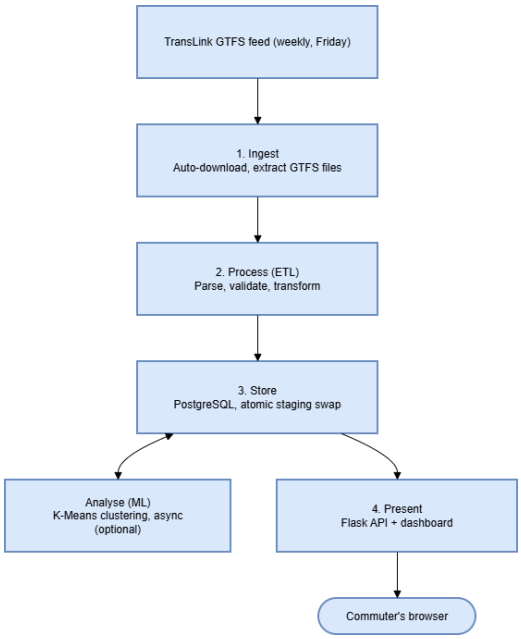


Figure 4: System Architecture Diagram, updated version with "Analyse (ML, optional)" label

- **ETL Pipeline Flowchart** — weekly cron trigger through feed-change check, validation, staging load, production swap, and optional clustering

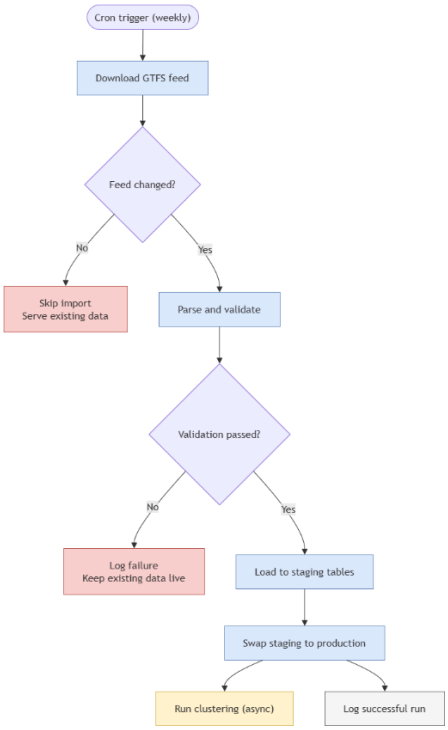


Figure 5: ETL Pipeline Flowchart

- **Use Case Diagram**: Commuter, Scheduler, Data Enthusiast, and Researcher/Planner actors and their use cases

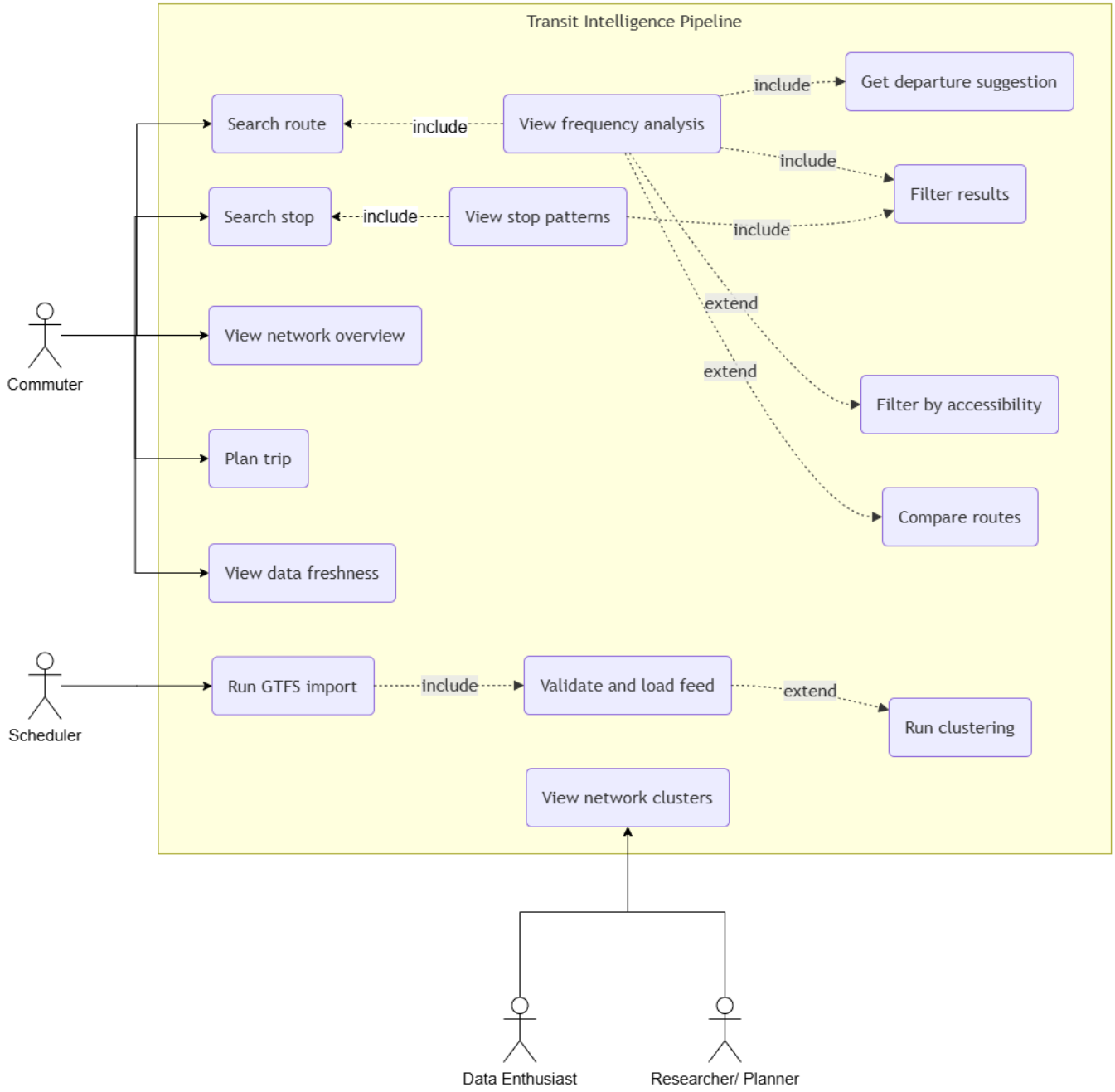


Figure 6: Use Case Diagram, corrected version with include/extend directions fixed

- **Route Analyser Sequence Diagram** and **Stop Explorer Sequence Diagram**

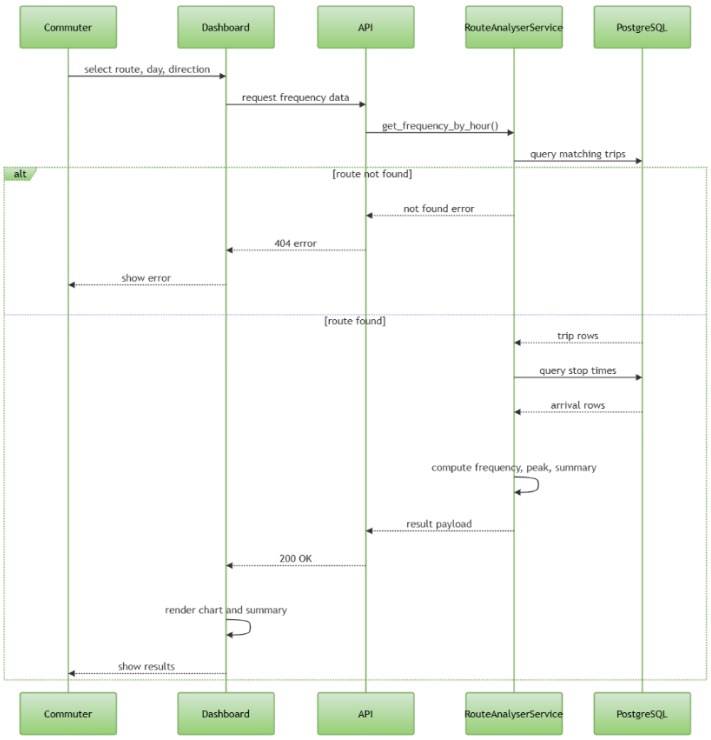


Figure 7: Route Analyser Sequence Diagram

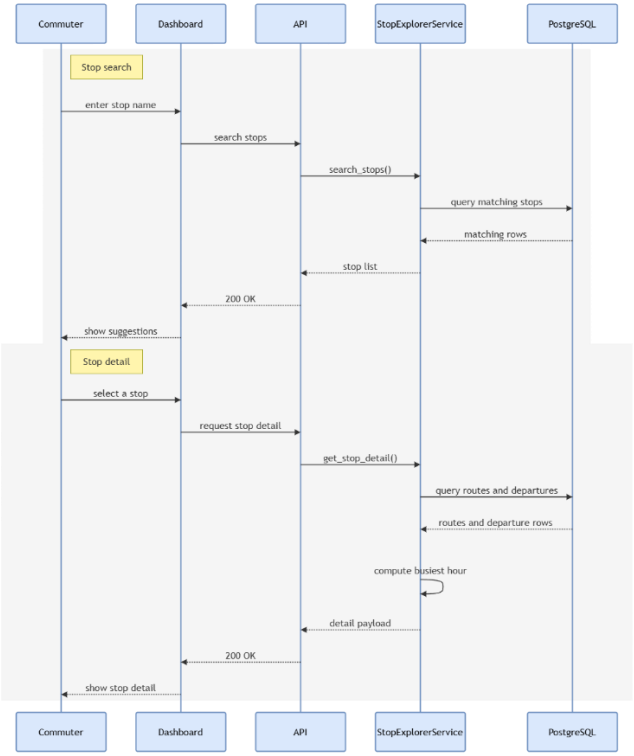
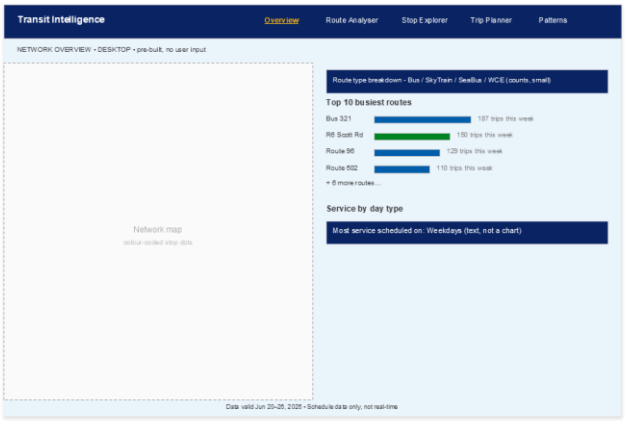


Figure 8: Stop Explorer Sequence Diagram

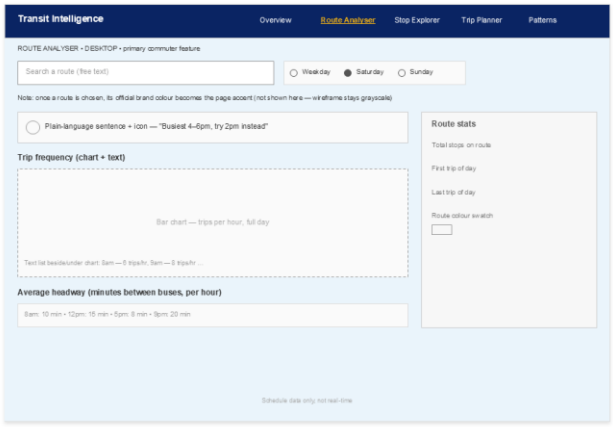
- **High-fidelity wireframes** for all five dashboard pages (Network Overview, Route Analyser, Stop Explorer, Trip Planner Lite, Network Patterns), in both Desktop and Mobile layouts

High-Fidelity Wireframe- Desktop

01 · Network Overview - Desktop

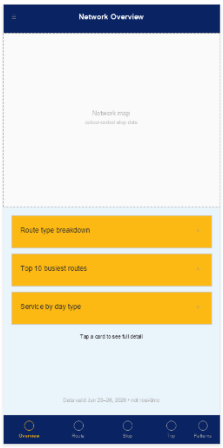


02 · Route Analyser - Desktop

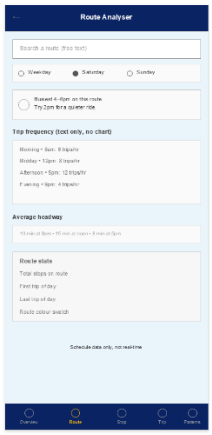


High-Fidelity Wireframe- Desktop

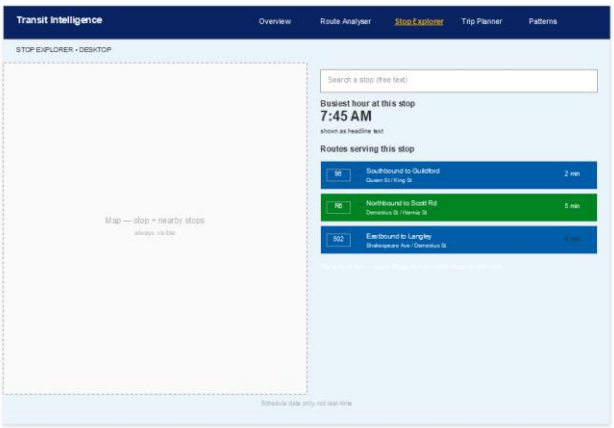
01 · Network Overview — Mobile



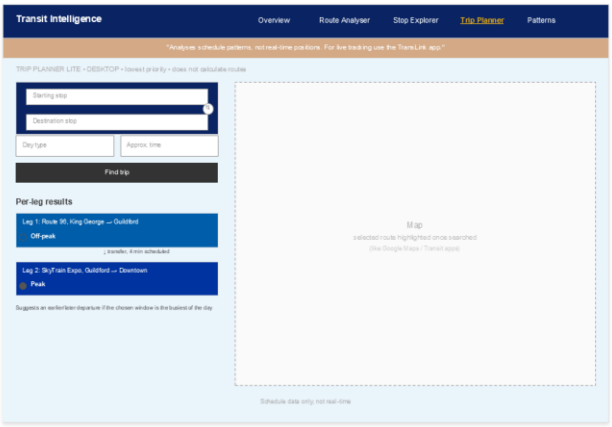
02 · Route Analyser — Mobile



03 · Stop Explorer - Desktop



04 · Trip Planner Lite - Desktop



05 · Network Patterns - Desktop

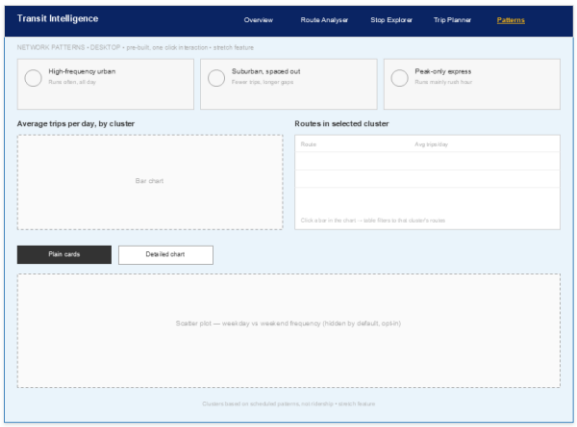
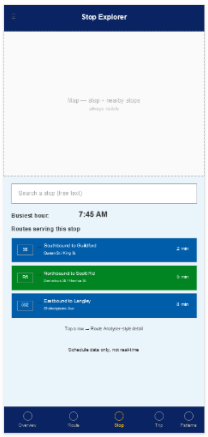


Figure 9: High-Fidelity Wireframes, (Desktop)

03 · Stop Explorer — Mobile



04 · Trip Planner Lite — Mobile



05 · Network Patterns — Mobile



Figure 10: High-Fidelity Wireframes, (Mobile)

Justification of Design Choices

The Route Analyser and its plain-language summary are the core of the system, supported by stakeholder feedback (commuters travelling with strollers, mobility aids, or elderly family members actively look for quieter windows), the Week 1/4/5 data validation confirming the underlying GTFS data is reliable, and literature showing frequency variation between peak and off-peak has real, measurable impact.

The weekly automated GTFS import and feed-change check exist because TransLink refreshes its feed every Friday; without automated ingestion, the system's data would go stale within days. The staging-to-production swap design ensures the dashboard never serves a partially loaded dataset if an import fails partway through.

Direction-aware filtering is included because the Week 4/5 GTFS file analysis found TransLink's non-standard directions.txt file, a feature not offered by any of the three competitor tools reviewed. Accessibility-aware filtering (stretch) exists because the wheelchair_accessible and wheelchair_boarding columns are already present in the dataset at no additional collection cost, directly supporting the accessibility use case identified through stakeholder feedback.

K-Means clustering was deliberately scoped as a stretch feature rather than core, since the system's core commuter value, schedule-frequency aggregation by route, stop, day, time, and direction, works without it.

4.0 Test Plan and Test Case Design

Since implementation begins in INFO 4290, testing will follow the same order as the build roadmap: data layer first, backend second, frontend third, with usability testing once the dashboard is functional. Testing will combine four types: data/pipeline testing, input handling, navigation flow, and usability testing.

Data/Pipeline Testing

Test Case ID	Scenario	Input	Expected Output	Requirement
TC001	Weekly GTFS auto-download	Friday cron trigger fires	System automatically downloads the TransLink GTFS Static ZIP file from the published URL	FR-01
TC002	Successful full import	Valid weekly GTFS ZIP containing routes,	All core GTFS files parse and load into PostgreSQL without errors	FR-02

		trips, stop_times, calendar, stops		
TC003	GTFS feed unchanged	Run weekly import when feed_info.txt dates match the last run	Skip re-import; existing data continues serving the dashboard	FR-03
TC004	Large file memory handling	Load stop_times.txt (87.78 MB) during import	File loads using Pandas column-selective loading, reducing memory usage	NFR-02
TC005	Feed not published	Simulate a Friday with no new GTFS feed available	System remains functional, serving the most recently loaded dataset until a new feed is available	NFR-05
TC006	Version control naming	Review design artefacts and source files in shared GitHub repository	All files version-controlled using consistent naming conventions agreed upon in Week 2	NFR-06

Input Handling

Test Case ID	Scenario	Input	Expected Output	Requirement
TC007	Route not found	Search "Route 9999" (nonexistent)	Display: "Route not found."	FR-04
TC008	Stop search with valid results	Search "King George"	Display list of matching stops	FR-05
TC009	Accessibility filter	Filter trips/stops using wheelchair_accessible and wheelchair_boarding columns	Only accessible trips/stops are returned	FR-12

Navigation Flow

Test Case ID	Scenario	Input	Expected Output	Requirement
TC010	Frequency by day/direction	Select Route 96, Sunday, Southbound	Display frequency by route, day type, time window, and direction	FR-06
TC011	Data refresh status display	Load any dashboard page after a successful import	Show date GTFS feed was last processed and its validity dates	FR-08
TC012	Dashboard filter combination	Filter by route, day of week, and time range together	Results narrow to match all three filters	FR-09
TC013	Disclaimer visibility	Load any dashboard page	Disclaimer stating results reflect scheduled service frequency only, not real-time ridership or crowding, is visible	FR-10
TC014	Service intensity chart renders	Load Route Analyser for a valid route and day type	Chart displays scheduled frequency patterns for that route and day type	FR-11
TC015	Route comparison	Compare scheduled service frequency across two routes	Frequency displayed side-by-side for both routes	FR-13
TC016	K-Means clustering display	Run K-Means clustering on route service behaviour	Results displayed on the Network Patterns dashboard page	FR-14
TC017	Dashboard load time	Load a core dashboard page on standard broadband	Page loads within 3 seconds	NFR-01

Usability Testing

Test Case ID	Scenario	Input	Expected Output	Requirement
TC018	Plain-language summary comprehension	Show plain-language summary screenshot to a test participant, no explanation given	Participant correctly interprets the recommendation	FR-07, NFR-04
TC019	Terminology consistency	Review UI copy, plain-language summaries, and design documents	Consistently use "scheduled service frequency" or "service intensity," never "busy," "crowded," or "demand" without qualification	NFR-03

Requirement Reference

#	Requirement	Priority/ Category
FR-01	The system shall automatically download the TransLink GTFS Static ZIP file from the published URL every Friday	Core
FR-02	The system shall parse and load all core GTFS files (routes, trips, stop_times, calendar, stops) into PostgreSQL without errors	Core
FR-03	The system shall check feed_info.txt after each download and skip re-import if the feed dates have not changed	Core
FR-04	The system shall allow users to search for a route by number or name	Core
FR-05	The system shall allow users to search for a stop by name and view its scheduled service patterns	Core
FR-06	The system shall display scheduled service frequency by route, day type (weekday/Saturday/Sunday), time window, and direction	Core
FR-07	The system shall generate a plain-language departure suggestion for every Route Analyser result	Core

FR-08	The system shall display a data refresh status indicator showing when the GTFS feed was last processed and its validity dates	Core
FR-09	The system shall support filtering by route, stop, day of week, time range, transit mode, and direction	Core
FR-10	The system shall display a disclaimer on every dashboard page clarifying that results reflect scheduled service frequency only, not real-time ridership or crowding	Core
FR-11	The system shall display service intensity charts visualizing scheduled frequency patterns for a selected route and day type	Core
FR-12	The system shall support filtering trips and stops by wheelchair accessibility using the wheelchair_accessible and wheelchair_boarding columns already present in the TransLink GTFS dataset	Stretch
FR-13	The system shall allow users to compare scheduled service frequency across two routes or two departure windows	Stretch
FR-14	The system shall apply K-Means clustering to group routes by service behaviour and display results on the Network Patterns dashboard page	Stretch
NFR-01	Core dashboard pages shall load within 3 seconds on a standard broadband connection	Performance
NFR-02	The stop_times.txt file (87.78 MB) shall be loaded using Pandas column-selective loading to reduce memory usage	Performance
NFR-03	All UI copy, plain-language summaries, and design documents shall consistently use "scheduled service frequency" or "service intensity", never "busy," "crowded," or "demand" without qualification	Usability
NFR-04	The plain-language departure recommendation shall be correctly interpreted by at least 6 out of 8 test participants without any explanation from the development team	Usability
NFR-05	The system shall remain functional if TransLink does not publish an updated feed on a given Friday, serving the most recently loaded dataset until a new feed is available	Reliability
NFR-06	All design artefacts and source files shall be version-controlled in a shared GitHub repository using consistent naming conventions agreed upon in Week 2	Maintainability

5.0 Peer Feedback and Improvements

A second peer review round was not required; therefore, this section summarizes the feedback received during Peer Review #1 and the resulting improvements.

Team 8 reviewed Progress Report 1 and rated it highly across all five evaluation criteria, scoring 5/5 for Clarity of Scope, Actionable Next Steps, and Overall Organization, and 4/5 for Relevance of Research and Identification of Risks. The reviewers noted that the project demonstrated strong technical research and validation, and that prioritizing core commuter value over machine learning complexity reflected sound project management.

Two specific, actionable suggestions were raised:

Feedback 1: Include a system architecture diagram earlier in the report, rather than waiting for a later deliverable.

Action taken: A simplified four-layer architecture diagram was added to the Midterm Report as an early, high-level version. This was then expanded into the full, detailed architecture diagram included in Progress Report 2 as Figure 4, showing the Ingest, Process (ETL), Analyse (ML), and Present layers in detail.

Feedback 2: Strengthen the risk section with user adoption and usability risks, not only technical risks.

Action taken: A new risk was added to the risk register covering low user adoption due to unfamiliar terminology or an unclear value proposition, specifically that commuters may not understand what "scheduled service frequency" means or may not see why the tool is more useful than apps they already use for real-time arrivals. This is mitigated through the plain-language summary feature and the Week 7+ user testing plan (8 participants, 6/8 pass threshold), along with onboarding copy that explicitly contrasts this tool with real-time apps.

These changes improved both the completeness of the system design and the clarity of how users are expected to interpret transit patterns.

6.0 Project Evaluation and Reflections

Progress Against Original Goals

The goal was to design a complete, implementation-ready pipeline connecting TransLink's GTFS data to a commuter dashboard. That goal was met — all core design deliverables are done: the database ERD, class diagrams, system architecture diagram, ETL flowchart, use case diagram, sequence diagrams, and high-fidelity wireframes for all five dashboard pages.

The scope changed in three ways since the proposal, all based on instructor feedback: K-Means clustering became an optional stretch feature instead of a core component, terminology was standardized around "scheduled service frequency," and user testing was added as a success metric. None of these changed the core direction of the project.

Lessons, Team Dynamics, and Project Planning

The division of labour set at the proposal, Inderpreet on technical design, Drishti on UI/UX and documentation, held through the semester and expanded naturally as the work required it.

The main lesson was the value of testing early. Testing the database design against real TransLink data before implementation caught real problems, extra columns, non-standard date formats, orphaned route references, midnight-crossing trip times, and a missing composite key, while they were still cheap to fix.

Remaining Risks and Next Steps

The risk register built up this semester, scope size, K-Means reliability, GTFS format changes, team coordination, and user adoption of unfamiliar terminology, carries into the INFO 4290 build phase, where it will be tested against a real implementation.

7.0 Conclusion

The TransLink Transit Intelligence Pipeline successfully establishes a complete design for transforming GTFS schedule data into commuter-focused insights. The finalized architecture, database design, ETL workflow, testing strategy, and dashboard prototypes provide a strong foundation for implementation in INFO 4290. Future work will focus on system development, performance testing, and validating the usefulness of departure recommendations through user testing.

8.0 Appendix

1. Figure 1: Database ERD
2. Figure 2: Domain Class Diagram
3. Figure 3: Service Importer Class Diagram
4. Figure 4: System Architecture Diagram
5. Figure 5: ETL Pipeline Flowchart
6. Figure 6: Use Case Diagram
7. Figure 7 : Route Analyser Sequence Diagram
8. Figure 8: Stop Explorer Sequence Diagram
9. Full Desktop Wireframes file: Desktop Wireframes.pptx
10. Full Mobile Wireframes file: Mobile Wireframes.pptx